ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Self-evolving agents store experience in persistent skills, but a higher task score leaves open what the skill has learned. We study whether the resulting benefit persists beyond the source task and which decisions in the skill explain its success or failure. We diagnose completed SkillEvolver, SPARK, and CoEvoSkills artifacts through repeated paired evaluations, controlled task variations, and inspection of skill contents and execution traces, using their construction procedures as context. Source-task gains recur across all three methods, while changed tasks expose different scopes of useful reuse. Contrasting cases relate reuse outcomes to how skills distinguish valid answers from plausible candidates and preserve the constraints that define a correct solution. The findings connect benchmark gains to persistent decision guidance and distinguish producing a useful skill, applying its guidance, and sustaining its benefit under task change.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Persistent agent skills offer a route to improvement without updating model parameters. Instructions, scripts, and supporting references make execution experience available to later runs. SkillEvolver, SPARK, and CoEvoSkills construct these artifacts through deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification [1–3]. Their outputs motivate the question in our title: when a system produces a higher-scoring skill, what useful behavior has become persistent?

The location of that improvement matters. A skill can preserve a decision rule that remains useful on new inputs, a choice tailored to the original instance, or a check that a later executor fails to apply. These artifacts may all help on the source task, yet lead to different outcomes when the task changes. The final score also combines artifact construction, execution, and verification. Understanding skill learning therefore requires following the stored guidance into the behavior it supports.

We ask two connected questions. **Does the benefit persist, and under which task changes?** Repeated source-task evaluations test whether paired utility recurs, while frozen-skill variants expose the demands that the same guidance can accommodate. **What in the skill and its use explains the observed outcomes?** Contrasting executions reveal whether decisive correctness criteria are available in the skill and applied to the new task. These questions connect the existence and scope of useful behavior with an account of how it is produced.

We examine three construction mechanisms through two standard-task runs per method and controlled changes to input representation, task instances, and coupled requirements. Each skill condition is paired with a no-skill execution of the same task. Signal-detection and scheduling cases then connect the aggregate patterns to decisions visible in the skills and execution traces. The completed artifact is the common object of diagnosis: its construction provides context, and its later use supplies the behavioral evidence.

All three methods yield positive mean source-task gains in both runs, but this agreement does not persist under task variation. SkillEvolver remains positive across all three change types. SPARK's benefit depends on the change type, and CoEvoSkills has negative mean gains in all three. The clearest separation occurs when the instance is replaced. The cases explain why executing a procedure is only part of the problem: the executor must also distinguish the required signal from misleading candidates and a valid schedule repair from an otherwise feasible schedule.

Our contribution is a diagnosis of what observed skill gains represent. Repeated paired evaluations establish recurring utility, controlled variants locate its scope, and artifact-level cases identify decisions associated with successful and failed reuse. Together, these results show how a score advantage can be interpreted through the guidance that survives construction and is actually used during execution.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

自进化智能体技能是否在学习技能?

匿名提交

摘要

自进化智能体将经验存储在持久技能中，但任务分数提高后，技能究竟学到了什么仍有待解释。我们研究收益能否在源任务之外持续，以及技能中的哪些决策解释了成功与失败。我们通过重复的配对评估、受控任务变化，以及技能内容和执行轨迹检查，诊断 SkillEvolver、SPARK 和 CoEvoSkills 的最终技能产物，并以其构建过程作为解释背景。三种方法的源任务收益均反复出现，但任务变化揭示了不同的有效复用范围。对照案例将复用结果与区分有效答案和候选答案、保留正确解所需约束的指导联系起来。这些发现连接了基准收益与持久决策指导，并区分产生有用技能、应用其指导和在任务变化后保持收益。

关键词—智能体技能，自进化，诊断评估，技能复用，泛化

1. 引言

持久技能为智能体提供了一条无需更新模型参数的改进路径。指令、脚本和参考资料让后续执行能够利用已有经验。SkillEvolver、SPARK 和 CoEvoSkills 分别通过基于部署的修订、经验证轨迹的蒸馏和协同进化验证来构建技能产物 [1–3]。这些产物引出了标题中的问题：系统生成得分更高的技能时，哪些有用行为得以持久保留?

那种改进的位置很重要。一个技能可以保留一个在新的输入上仍然有用的决策规则、一个针对原始实例的定制选择，或一个后续执行者未能应用的检查。这些工件可能都在源任务上有所帮助，但在任务变化时会导致不同的结果。最终得分还结合了工件构建、执行和验证。因此，理解技能学习需要追踪存储的指导进入它支持的行为。

我们提出两个相关问题。收益是否持续，在哪些任务变化下持续？重复的源任务评估检验配对效用是否反复出现，冻结技能的变体评估揭示同一指导能够满足哪些需求。技能及其使用中的哪些决策解释了观察结果？对照执行可以揭示技能是否提供关键的正确性标准，以及执行者是否将它用于新任务。这两个问题将有用行为的存在及范围，与行为如何产生的解释联系起来。

我们通过每种方法的标准任务运行两次和输入表示、任务实例以及耦合需求的控制变化，来检验三种构建机制。每种技能条件都与相同任务的无技能执行配对。信号检测和调度案例将总体模式与技能和执行轨迹中可见的决策联系起来。完成的工件是诊断的共同对象：它的构建提供了背景，而它的后续使用则提供了行为证据。

三种方法在两次运行中均有正向源任务平均增益，但任务变化打破了这种一致性。SkillEvolver 在全部三类变化下保持正向收益。SPARK 的收益取决于变化类型，CoEvoSkills 则在三类变化下均为负向均值。实例替换时的分化最明显。案例说明，执行程序只是问题的一部分：执行者还须区分目标信号与误导候选，以及有效的调度修复与仅满足可行性的排程。

本文贡献在于诊断观察到的技能收益代表什么。重复配对评估检验效用是否反复出现，受控变体界定其范围，技能产物层面的案例则识别与复用成功及失败相关的决策。这些结果共同说明，分数优势可以通过构建后保留、并在执行中实际使用的指导来解释。

2. 问题表述

2.1. 技能修订与评估

在修订轮次 r 中，优化器更新了当前的持久技能组件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 s_r 是当前技能，τ_r 是执行轨迹，而 f_r 是来自执行和评估的反馈。运算符 O 表示方法完整的更新路径，包括其原生合并、验证、选择或采用机制。一个被拒绝或从未被采用的生成候选者不是 s_{r+1}。

随后，智能体使用冻结技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

z_r 表示模型采样、工具使用和环境状态，而 o_r 是产生的输出工件或状态变化。验证器将输出映射到分数：

$$y_r = V(x, o_r). \tag{3}$$

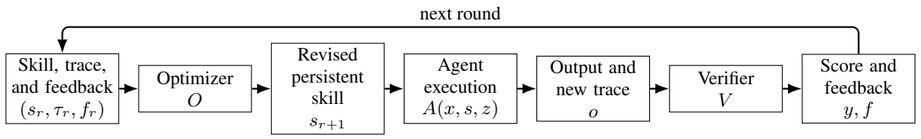


Fig. 1. The skill-evolution lifecycle. Construction produces a persistent artifact; deployment turns its guidance into behavior; verification assigns the observed score. We connect these stages through paired utility, frozen-skill task variation, and inspection of skill use.

2.2. From Score Gains to Skill Behavior

The score $V(x, A(x, s, z))$ records the outcome of a skill used by an executor and assessed by a verifier. Comparing a completed skill with no skill measures its utility at deployment. Comparing the adopted final skill with its actual parent addresses the improvement associated with revision. The experiments here follow the first contrast across repeated runs and changed tasks.

For each source task x, we freeze its completed skill s_x and evaluate it on a variant $T_k(x)$ alongside a no-skill execution. The skill remains unchanged, while the executor can reinterpret the input, recompute a solution, and adapt its actions. A positive paired difference on the variant indicates that the stored guidance remains useful under that change. The variation types distinguish the decisions that can be reused from those that the executor must reconsider.

The attribution question follows the same artifact into execution. We examine which candidates its rules distinguish, which constraints its checks preserve, and where the observed behavior departs from that guidance. The construction procedures describe the feedback from which the guidance is produced. Reading these stages together connects a score change to a specific decision, rather than treating the presence of a skill file as evidence that its instructions were used effectively.

3. DOES SKILL UTILITY PERSIST BEYOND THE SOURCE TASK?

We first test whether completed skills provide a recurring benefit on their source tasks. We then change the task while keeping its skill frozen to determine which demands that benefit survives. The paired no-skill execution provides the reference for both comparisons.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Positive Source-Task Utility in Both Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. The paired comparison reveals recurring utility that absolute skill-condition scores alone would obscure.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. The gain therefore reflects both the size and frequency of task-level changes. The frozen-skill variants test whether the useful guidance behind these source-task outcomes survives a different task demand.

Method	Run	N	No skill	Skill	Δ	I/R/T
SkillEvolver	1	81	30.20	55.81	+25.61	27/6/48
	2	81	37.32	62.34	+25.02	26/3/52
SPARK	1	84	17.82	19.66	+1.84	9/8/67
	2	84	14.37	20.81	+6.45	13/5/66
CoEvoSkills	1	85	42.21	52.44	+10.24	17/9/59
	2	85	23.54	33.31	+9.76	17/10/58

Table 1. Two runs per method on SkillsBench. Mean verifier rewards and paired gain Δ use a 0–100 scale. I/R/T counts improved, regressed, and tied tasks.

3.3. Task Variation Tests What Remains Useful

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. The corresponding decision demands are to remap the input, recompute a solution for a replacement instance, and accommodate changed correctness requirements. A frozen skill must support these adaptations through guidance already present in the artifact.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. The improvement and regression counts clarify the near-zero SPARK mean: it combines changes in opposite directions, rather than uniformly neg-

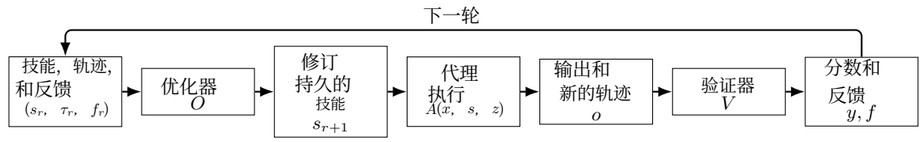


图1。技能进化生命周期。构建产生一个持久的工件；部署将其指导转化为行为；验证分配观察到的分数。我们通过配对效用、冻结技能任务变异以及检查技能使用来连接这些阶段。

2.2. 从分数收益到技能行为

分数 $V(x, A(x, s, z))$记录了执行者使用并由验证器评估的技能的结果。将完成的技能与无技能进行比较，可以衡量其在部署时的效用。将采用的最终技能与其实际父技能进行比较，可以解决与修订相关的改进问题。这里的实验遵循第一种对比，即在重复运行和任务变化中进行比较。

对于每个源任务 x，我们冻结其完成的技能 s_x，并在一个变体 $T_k(x)$ 上对其进行评估，同时进行无技能执行。技能保持不变，而执行者可以重新解释输入，重新计算解决方案，并调整其动作。在变体上的正向配对差异表明，存储的指导在那种变化下仍然有用。变化类型区分了可以重用的决策和执行者必须重新考虑的决策。

归因问题遵循相同的工件进入执行。我们检查其规则区分了哪些候选，其检查保留了哪些约束，以及观察到的行为与该指导有何不同。构建程序描述了指导产生的反馈。一起阅读这些阶段将分数变化与特定决策联系起来，而不是将技能文件的存在视为其指令被有效使用的证据。

3. 技能效用是否超出源任务?

我们首先检验最终技能是否反复对源任务产生收益，再冻结技能并改变任务，考察收益能够适应哪些需求。配对的无技能执行为这两种比较提供参照。

3.1. 实验设置

我们在 SkillsBench [4] 上评估 SkillEvolver [1]、SPARK [2] 和 CoEvoSkills [3]，每种方法运行两次。每次将技能条件与同题无技能执行配对。同一方法两次运行使用相同任务集：SkillEvolver 81题，SPARK 84题，CoEvoSkills 85题。

我们报告0–100分制的平均验证器奖励，以及技能与无技能条件的平均同题差值。对包含 N 个任务的运行，配对增益为

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

我们还统计了具有正、负和零差异的任务数量。

3.2. 两次运行均有正向源任务效用

三种方法两次均有正向平均配对增益（表1）。SkillEvolver 提高25.61和25.02分，CoEvoSkills 提高10.24和9.76分，SPARK 提高1.84和6.45分。SkillEvolver 和 CoEvoSkills 的无技能分数虽变化较大，增益却相近。配对比较揭示了技能条件的绝对分数会掩盖的重复效用。

逐题结果也有差异。SkillEvolver 每次运行的改进数至少为退化数的四倍；CoEvoSkills 两次均改进17题，分别退化9题和10题。SPARK 首次的改进与退化数接近，第二次为13次改进、5次退化。增益因此同时反映逐题变化的幅度和频率。冻结技能的变体评估检验这些收益背后的指导能否持续适应不同任务需求。

Method	Run	N	No skill	Skill	Δ	I/R/T
SkillEvolver	1	81	30.20	55.81	+25.61	27/6/48
	2	81	37.32	62.34	+25.02	26/3/52
SPARK	1	84	17.82	19.66	+1.84	9/8/67
	2	84	14.37	20.81	+6.45	13/5/66
CoEvoSkills	1	85	42.21	52.44	+10.24	17/9/59
	2	85	23.54	33.31	+9.76	17/10/58

表1. SkillsBench 上每种方法的两次运行。平均验证器奖励及配对增益 Δ 均按0–100分制报告。I/R/T 分别为改进、退化和持平的任务数。

3.3. 任务变化测试什么仍然有用

我们对每个源任务技能应用于受控变体，不再进行进一步进化。输入表示变化（P1）会改变名称、列排列或模式；任务实例变化（P2）会替换数据或实例结构；耦合需求变化（P3）会改变或添加必须一起满足的需求。相应的决策需求是重新映射输入、为替换实例重新计算解决方案，并适应变化后的正确性需求。一个冻结的技能必须通过工件中已有的指导来支持这些适应性。

完整记录包含222个变体，以及无技能和三种方法共886条条件级结果。表2汇总历史评估与补跑：SkillEvolver 覆盖222个变体，SPARK 和 CoEvoSkills 各覆盖221个。每个技能条件均与同一变体的无技能结果配对。

源任务改进不等于有效泛化。三种方法两次均提高源任务平均奖励，变体结果却分化。SkillEvolver 的平均增益为11.94分，SPARK 的 +0.65 接近零，CoEvoSkills 则低于无技能基线5.01分。SkillEvolver 的改进数多于退化数（47比20），另两种方法则相反。SPARK 接近零的均值汇集了方向相反的变化。

ligible effects. SkillEvolver shows both a larger positive mean and more frequent improvement than regression.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 (+14.56). P2 is also where SPARK and CoEvoSkills decline most (−5.23 and −9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. P2 requires choices to be recomputed from new data or a new instance structure. A procedure that remains useful after renaming inputs can therefore lose its benefit when the underlying decision changes. SkillEvolver's positive gains across P1–P3 show the broadest observed persistence of utility, while CoEvoSkills' negative means show that source-task guidance can become counterproductive after the task changes.

A. Overall utility on task variants					
Method	N	**No skill**	**Skill**	Δ	**I/R/T**
SkillEvolver	222	41.87	53.81	+11.94	47/20/155
SPARK	221	41.61	42.25	+0.65	32/37/152
CoEvoSkills	221	41.61	36.60	−5.01	26/41/154

B. Utility by task change: (N) Δ			
Method	**P1**	**P2**	**P3**
SkillEvolver	(77) +11.85	(68) +14.56	(77) +9.72
SPARK	(77) +2.00	(68) −5.23	(76) +4.53
CoEvoSkills	(77) −3.59	(68) −9.51	(76) −2.42

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Each method is paired with its same-variant no-skill baseline. P1/P2/P3 denote representation, instance, and requirement changes. Panel B gives variant count N and paired gain Δ; scores and I/R/T follow Table 1.

4. LOCATING SUCCESSFUL AND FAILED REUSE

The aggregate results locate differences in scope; the cases locate the decisions behind them. Signal detection tests how a skill distinguishes the required signal from plausible candidates. Scheduling tests whether a skill preserves the conditions that define an acceptable repair. In each case, we relate the rules in the artifact to their observed use.

4.1. Distinguishing a Candidate from a Valid Answer

In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. Both skills fail the replacement-instance P2 variant.

The difference is visible in what the skills ask the executor to check. SkillEvolver separates candidate discovery, signal diagnosis, and refinement. Its guidance addresses misleading peaks, transit-like durations, and harmonic relationships. CoEvoSkills also includes validation, including phase-fold, odd/even, and residual checks, but its use in these variants does not resolve the misleading candidates. The presence of a checking routine is therefore less informative than whether its criteria distinguish the alternatives that matter on the changed task. SkillEvolver's P1 and P3 successes show this guidance being useful under changed inputs and requirements. Its P2 failure locates a different boundary within the same task family: the retained criteria do not resolve that replacement instance, despite a positive P2 average across the broader evaluation.

4.2. Preserving the Rules That Define a Correct Repair

In scheduling, the SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using CoEvoSkills yields an otherwise feasible schedule that fails a required local-minimality check on the baseline-relative repair. Feasibility checks alone do not capture this local-minimality requirement.

The SkillEvolver skill explicitly retains local minimality and baseline-relative repair rules. These rules specify how an acceptable repair differs from an arbitrary feasible schedule, and remain relevant after the input names or scheduling instance change. The CoEvoSkills skill covers several structural and feasibility checks, yet the failed deployment misses the decisive repair condition. The contrast concerns whether the decisive repair condition is preserved in guidance and enforced during its use. SPARK's success across the same three variants also shows that this form of successful reuse is not exclusive to one method.

4.3. Construction Feedback and Skill Content

The construction procedures help interpret how these decision criteria enter a skill. SkillEvolver compares successful and failed deployments of a candidate skill and patches missing or misleading guidance. Its independent audit also checks reliance on training-instance constants [1]. This exposes failures in using the skill and encourages rules that a later executor can apply to a different input. The scheduling and signal-detection skills reflect that emphasis by retaining criteria for checking a repair or discriminating between candidate signals.

SPARK grounds skill distillation in environment-verified trajectories, preserving execution evidence, checks, and lessons [2]. Its scheduling result shows that such evidence can support successful reuse. Its mixed aggregate profile raises a separate question about abstraction: a verified execution supplies evidence for what worked on the source task, while changed instances require determining which choices should be retained and which should be recomputed. The observed decline on P2 makes that distinction especially relevant to interpreting its results.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier, while the ground-truth oracle returns an opaque pass/fail signal [3]. This makes the surrogate's coverage of correctness conditions relevant to the guidance retained in the skill. This provides a plausible account of the scheduling failure: the skill preserves many checks, but successful reuse still depends on a condition those checks did not enforce in that deployment. Across the cases, feedback coverage and abstraction from the source instance are candidate explanations for the different scopes of utility.

5. INTERPRETING GAINS AS EVIDENCE OF SKILL LEARNING

5.1. A Useful Source Skill Can Have a Narrow Behavioral Scope

The standard-task and variant results identify two properties of the completed artifacts. Source-task utility recurs for all three methods, but its scope differs under controlled changes. SkillEvolver's positive gains across P1–P3 support useful reuse under each tested

这并非每个变体都变化很小。SkillEvolver 同时具有更大的正向均值和多于退化的改进数。

实例替换带来的分化最明显。SkillEvolver 在三类变化下均为正，P2 增益最大（+14.56）。SPARK 和 CoEvoSkills 在 P2 上下降也最明显（−5.23 和 −9.51）。SPARK 在表示和需求变化下保持正向均值，底层实例变化后却失去优势。P2 要求根据新数据或实例结构重新计算决策，因此重命名输入后仍然有用的程序，在决策本身变化后可能失效。SkillEvolver 在 P1–P3 上均有正向收益，呈现最广的效用持续范围；CoEvoSkills 的负向均值说明，源任务指导在任务变化后可能适得其反。

A. Overall utility on task variants					
Method	N	**No skill**	**Skill**	Δ	**I/R/T**
SkillEvolver	222	41.87	53.81	+11.94	47/20/155
SPARK	221	41.61	42.25	+0.65	32/37/152
CoEvoSkills	221	41.61	36.60	−5.01	26/41/154

B. Utility by task change: (N) Δ			
Method	**P1**	**P2**	**P3**
SkillEvolver	(77) +11.85	(68) +14.56	(77) +9.72
SPARK	(77) +2.00	(68) −5.23	(76) +4.53
CoEvoSkills	(77) −3.59	(68) −9.51	(76) −2.42

表2. 任务变化下的技能效用，描述性汇总历史评估与补跑结果。每种方法均与同一变体的无技能基线配对。P1/P2/P3 分别为表示、实例和需求变化。B部分给出变体数 N 与配对增益 Δ；分数及 I/R/T 定义同表1。

4. 定位复用成功与失败的环节

总体结果定位了范围差异；案例定位了其背后的决策。信号检测测试一个技能如何区分所需信号与合理的候选者。调度测试一个技能是否保持定义可接受修复的条件。在每个案例中，我们将工件中的规则与其观察到的使用情况联系起来。

4.1. 区分候选答案与有效答案

在系外行星检测中，SkillEvolver 技能在输入列重排（P1）和要求两个信号（P3）时均成功。CoEvoSkills 两者均失败：将恒星调制报告为目标信号，并将一个信号及其谐波视为两个独立信号。两种技能均未通过替换实例的 P2 变体。

差异体现在技能要求执行者检查什么。SkillEvolver 将候选发现、信号诊断和精炼分开，指导执行者处理误导峰值、符合凌日的持续时间及谐波关系。CoEvoSkills 也包含相位折叠、奇偶及残差检查，但执行时未能排除误导候选。检查流程的存在不如其标准能否区分关键候选更有解释力。SkillEvolver 在 P1 和 P3 成功，说明指导适用于变化后的输入和需求。其 P2 失败则定位了同一任务家族内的不同边界：保留的标准未能解决那个替换实例，尽管在更广泛的评估中P2平均值是正的。

4.2. 保留定义正确修复的规则

在调度中，SkillEvolver 技能和 SPARK 技能对三种变体均成功；而协同进化技能（CoEvoSkills）技能仅对 P3 成功。在 P1 中，使用协同进化技能（CoEvoSkills）会得到一个原本可行但未通过基线相对修复中所需局部最小性检查的调度。仅通过可行性检查无法捕捉到这一局部最小性要求。

SkillEvolver 技能明确保留了局部最小性和基线相对修复规则。这些规则指定了可接受的修复与任意可行调度的区别，并且在输入名称或调度实例变化后仍然相关。协同进化技能（CoEvoSkills）技能涵盖了几种结构和可行性检查，但失败的部署遗漏了决定性修复条件。这种对比涉及指导中是否保留决定性修复条件，以及在使用过程中是否强制执行。SPARK 在相同三种变体中的成功也表明，这种形式的成功重用并非独属于某一种方法。

4.3. 构造反馈与技能内容

构造流程有助于解释这些决策标准如何进入技能。
SkillEvolver 比较候选技能的成功和失败部署，并修补缺失或误导性的指导。其独立的审计还会检查对训练实例常量 [1] 的依赖。这暴露了使用技能的失败，并鼓励后续执行者可以应用于不同输入的规则。调度和信号检测技能通过保留检查修复或区分候选信号的标准，反映了这种侧重。

SPARK 从经环境验证的轨迹中蒸馏技能，保留执行证据、检查和经验教训 [2]。调度结果说明这些证据能支持成功复用。
总体结果有升有降，提出了抽象问题：验证过的执行说明哪些做法在源任务上有效；实例变化后，仍须决定哪些选择应保留、哪些应重新计算。P2 上的下降使这一区分对理解结果尤为重要。

CoEvoSkills 从协同进化的替代验证器获取详细修订反馈，而真实评测器仅返回不透明的通过或失败信号 [3]。因此，替代验证器对正确性条件的覆盖范围与技能保留的指导相关。这为调度失败提供了一个可能解释：技能虽有多个检查，复用仍依赖此次执行中未落实的条件。反馈覆盖范围和对源实例的抽象程度，是解释不同效用范围的候选原因。

5. 将收益解释为技能学习的证据

5.1. 有用的源技能可以具有狭窄的行为范围

标准任务与变体结果揭示了最终技能的两个属性。三种方法的源任务效用均反复出现，但其适用范围随任务变化而分化。
SkillEvolver 在 P1–P3 上均有正向增益，支持各类受测变化下的有效复用。

change type. SPARK's positive P1 and P3 means alongside a negative P2 mean locate a more specific difficulty: adapting the representation or requirements can preserve its benefit, while replacing the underlying instance can remove it. CoEvoSkills' negative means show that guidance useful on the source task can interfere with a changed task.

Aggregate scope also contains local boundaries. SkillEvolver has the strongest P2 mean yet fails the replacement-instance signal-detection case. SPARK is nearly neutral across variants but succeeds on all three scheduling changes. These observations locate success and failure at the interaction between a skill and a task demand. The aggregate comparison reveals where to investigate; the case explains which decision remains useful or breaks.

5.2. Correctness Criteria Must Survive Construction and Use

The two cases expose distinct points where reuse can break. In signal detection, a procedure produces plausible candidates, but success depends on identifying which candidate satisfies the task. In scheduling, an output can satisfy structural feasibility while violating the rule that defines an acceptable repair. Reusable guidance must preserve this relationship between the procedure and its correctness criteria. The procedure's availability alone does not ensure that a later executor makes the required distinction.

This explains why inspecting the artifact and its execution together matters. CoEvoSkills contains validation checks, yet the observed executions miss decisive conditions. SkillEvolver's successful cases combine explicit decision criteria with their application to changed inputs. The contrast is between guidance as written and guidance as used: retained checks contribute only when they cover the relevant condition and are enforced during execution.

The lifecycle perspective also makes feedback informative. Deployment failures can reveal where instructions are ambiguous, verified trajectories can supply evidence for a successful procedure, and surrogate feedback can identify conditions to check. Each construction mechanism must convert that evidence into guidance that a later executor can apply. Our results locate the empirical question at this conversion and subsequent use: which decisions are preserved, when do they remain appropriate, and what happens when the executor follows them? This links the reported score back to the behavior that a persistent skill is intended to improve.

6. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and interpret the resulting behavior through its retained guidance.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study connects these evaluation levels to the identification of skill learning: repeated outcomes characterize completed-skill utility, task changes test its persistence, and skill-level examples explain the decisions behind contrasting outcomes. The unit of interpretation is the persistent artifact in use.

7. CONCLUSION

Our diagnosis connects score gains from self-evolving skills to the behavior carried by their persistent artifacts. All three methods provide recurring source-task utility, while controlled task changes reveal sharply different scopes of reuse. SkillEvolver sustains positive average gains across representation, instance, and requirement changes; the other methods show more selective or adverse reuse. Contrasting cases locate the difference in decisions that distinguish valid answers and preserve repair constraints, including whether the executor applies the checks present in a skill. These results make the learning question concrete: experience must become useful guidance, and that guidance must continue to support correct decisions when the task changes. Evaluating skill learning therefore requires connecting construction, stored content, and observed use, with the benchmark score recording the outcome of that process.

SPARK 在 P1 和 P3 上为正、P2 上为负，定位了更具体的困难：调整输入表示或需求可以保持收益，替换底层实例却可能消除收益。CoEvoSkills 的负向均值说明，对源任务有用的指导可能妨碍变化后的任务。

整体适用范围中也存在局部边界。SkillEvolver 的 P2 均值最强，却未通过替换实例的信号检测案例。SPARK 的整体变体收益接近零，却通过全部三个调度变体。成功与失败因此取决于技能与具体任务需求的交互：整体比较指出调查方向，案例解释哪些决策有效、哪些失效。

5.2. 在构建与使用中保留正确性标准

这两个案例暴露了可重用性可能被破坏的不同点。在信号检测中，一个程序会产生合理的候选者，但成功取决于识别哪个候选者满足任务。在调度中，一个输出可以满足结构可行性，同时违反定义可接受修复的规则。可重用指导必须保持程序与其正确性标准之间的关系。程序的可用性本身并不能确保后来的执行者做出所需的区分。

这解释了为什么检查工件及其执行一起很重要。CoEvoSkills 包含验证检查，但观察到的执行遗漏了决定性条件。SkillEvolver 的成功案例结合了明确的决策标准及其对更改输入的应用。对比在于写入的指导和使用中的指导：保留的检查只有在覆盖相关条件并在执行期间强制执行时才起作用。

生命周期视角使反馈也成为诊断依据。部署失败揭示指令不明确之处，经验证的轨迹提供成功过程的证据，替代验证器反馈指出需要检查的条件。每种构建机制都须将这些证据转化为后续执行者可用的指导。我们的结果将研究问题定位在这种转化及使用过程：哪些决策得以保留，何时仍然适用，以及执行者遵循它们时会发生什么？

这将报告的分数与持久技能旨在改进的行为联系起来。

6. 相关工作

从经验中学习持久技能。 SkillEvolver、SPARK 和 CoEvoSkills 通过不同的构建循环使经验持久化：基于部署的修订、验证轨迹的蒸馏和协同进化的验证，分别 [1–3]。这些方法促使我们关注传递给后续执行者的工件。我们研究该工件在源任务变化后的有用性如何变化，并通过其保留的指导来解释其行为。

评估技能效用与使用。SkillsBench 覆盖多样任务，SWE-Skills-Bench 研究软件工程中的技能使用 [4,5]。SkillLearnBench 评估真实任务上的持续技能生成 [6]。《从原始经验到技能使用》区分经验生成、技能提取与下游使用 [7]。我们将这些评估层面与技能学习的识别联系起来：重复结果刻画最终技能效用，任务变化检验其持续性，技能层面的案例解释不同结果背后的决策。解释的对象是执行中使用的持久技能产物。

7. 结论

我们的诊断将自进化技能的分数收益与持久技能产物承载的行为联系起来。三种方法均表现出反复出现的源任务效用，受控任务变化则揭示明显不同的复用范围。SkillEvolver 在表示、实例和需求变化下均保持正向平均收益，其他方法则表现出更局部或负面的复用效果。对照案例将差异定位到区分有效答案和维持修复约束的决策，包括执行者是否落实技能已有的检查。这使学习问题具体化：经验须转化为有用指导，且任务变化后仍须支持正确决策。因此，评估技能学习需要连接构建过程、存储内容和实际使用，基准分数记录了这一过程的结果。

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.